

LECTURE 7

Inter-Process Communication

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Why cooperating processes need a way to communicate at all
- Shared memory: fast, but synchronization becomes the processes' own responsibility
- Message passing: send()/receive(), direct vs. indirect, synchronous vs. asynchronous, buffering
- Sockets: an endpoint for communication, identified by an IP address and a port
- Remote Procedure Calls (RPC): calling a procedure on another machine as if it were local

Why Processes Need to Communicate

Most processes run completely independently and never need to talk to each other. But cooperating processes are explicitly designed to work together on a shared task, each contributing part of the result.

- A shell pipeline like `cat log.txt | grep ERROR | sort` chains three processes, where each one's output becomes the next one's input
- A producer/consumer pair — one process generates data, a second consumes it — common from print spoolers to streaming media pipelines
- Both need the same capability: a way to move data from one process's private address space into another's, safely and under OS supervision

Two IPC Models

There are two fundamentally different ways to provide that capability:

Shared memory

The OS sets up one region both processes can access directly; the processes do everything else themselves

Message passing

The OS mediates every exchange via `send()/receive()`; no memory is ever directly shared

Shared Memory

The OS sets up a single region of memory that multiple processes can map into their own address spaces. Once that mapping exists, those processes read and write it directly, exactly as if it were ordinary local variables.

- No further OS involvement in each individual read or write — which is exactly why this model is fast
- After the initial setup, communication is just memory access, with none of the overhead of a system call per message

Speed, at a Real Cost

The OS steps aside once the shared region exists. The processes themselves are entirely responsible for coordinating their access to it. If a producer is mid-write while a consumer reads, the consumer can see a half-written, inconsistent value — nothing about the mechanism itself prevents that. Making this safe gets a whole unit of its own later in this course: race conditions, mutual exclusion, semaphores, and monitors.

Message Passing

Processes never directly touch each other's memory. The OS provides two primitives, `send(message)` and `receive(message)`, and every exchange goes through the OS, which copies the message from sender to an OS-managed channel and out again to the receiver.

- Slower than shared memory — every message involves at least one trip through the OS (usually two copies of the data)
- Considerably simpler to get right: the OS mediates every exchange, so there's no way to see another process's half-finished write
- The same `send()/receive()` interface works whether the two processes are on the same machine or opposite sides of a network

Direct vs. Indirect Communication

Direct communication

`send(P2, msg)` — names the receiving process explicitly, one link per pair

Indirect communication

`send(mailbox, msg)` — sender and receiver need only agree on a shared mailbox or port

- Synchronous vs. asynchronous — blocking `send()/receive()` pause the caller until the other side has acted; non-blocking versions return immediately
- Automatic vs. explicit buffering — the OS can queue pending messages itself up to some capacity, or the application can manage that queue; zero-capacity buffering forces synchronous send/receive

Shared Memory vs. Message Passing, Side by Side

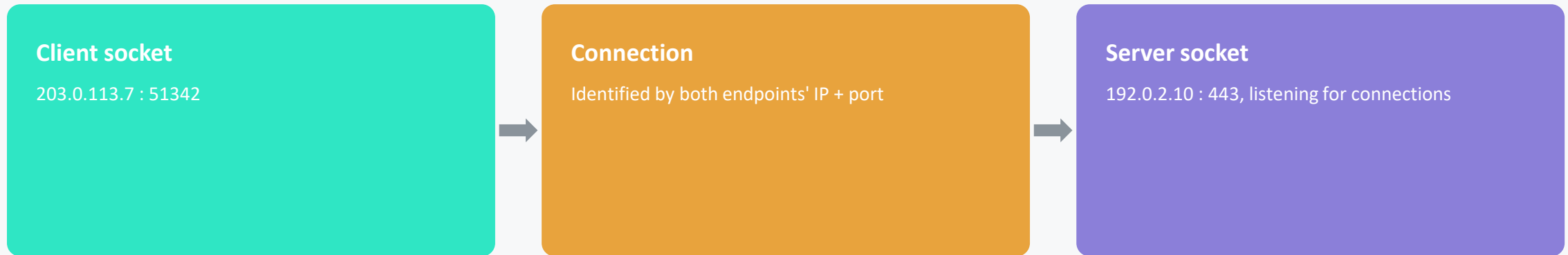
	Shared memory	Message passing
Speed	Fast — direct memory access after setup	Slower — every message is copied through the OS
Ease of correctness	Hard — processes must synchronize access themselves	Easier — the OS mediates every exchange
Works across machines?	No — requires physically shared memory	Yes — the same model scales naturally to a network
Best suited for	Tightly coupled processes on one machine, large data volumes	Loosely coupled or distributed processes, where safety matters more than raw speed

Sockets

A socket is an endpoint for communication, identified by the combination of an IP address and a port number. The IP address picks out a specific machine; the port number picks out a specific process or service on that machine willing to communicate.

- The client-server pattern is built directly on sockets: a server creates a socket and listens on a well-known port, clients create their own sockets and connect
- Once connected, both sides exchange data using essentially the same `send()/receive()`-style operations message passing already introduced
- A socket is, at its core, message passing extended across a network — not a third, unrelated mechanism

Client-Server Communication over Sockets



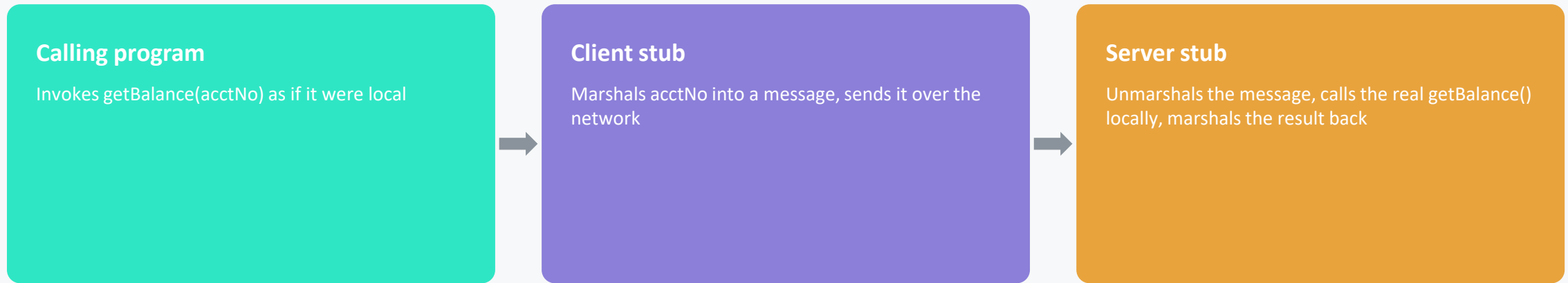
A port lets one machine run many independent services. Without ports, a single IP address could only ever run one network service at a time. Ports let the same machine run a web server on port 443, a mail server on port 25, and an SSH daemon on port 22 simultaneously, each receiving exactly the traffic addressed to its own port.

Remote Procedure Calls (RPC)

Sockets give a raw channel for exchanging bytes — but a programmer usually wants to call a function and get a result back, the same way they'd call any local function. An RPC lets a program call a procedure that actually executes on a different machine, while looking and feeling like an ordinary local call.

- RPC achieves this with stubs — small pieces of generated code on both sides of the call
- A client stub packages the arguments into a message (marshalling) and sends it over the network
- A server stub unmarshals the message, calls the actual procedure locally, and marshals the result back

An RPC Call, Traced Through Its Stubs



RPC is message passing with a procedure-call interface wrapped around it. Underneath the stubs, it's ordinary message passing (very often carried over sockets) between two processes on different machines. RPC's only addition is convenience — hiding the marshalling and network call behind something that looks syntactically identical to a local function.

Key Takeaways

- Cooperating processes — like a shell pipeline or a producer/consumer pair — need a deliberate, OS-provided way to exchange data across the isolation that normally separates processes
- Shared memory is fast because the OS steps aside after setup, but that speed means the processes themselves must synchronize their own access
- Message passing mediates every exchange through the OS via `send()/receive()`, trading some speed for safety and for working naturally across a network
- A socket is a communication endpoint identified by an IP address and a port, and underlies the familiar client-server pattern
- RPC lets a program call a procedure on a remote machine as if it were local, using client and server stubs to marshal and unmarshal parameters — built on top of ordinary message passing